

JSON and Nested Structured Data

Day 5 • Block 36 • 30 minutes teaching + 30 minutes exercises



JSON gives dictionaries and lists a portable text shape.

Key shift

Today we store structured Python data in a format other programs can read.

What Students Should Be Able to Do

The goal is not memorizing syntax. It is connecting files to Python structures.

1 Explain how JSON maps to Python dictionaries and lists

2 Load JSON files with `json.load()`

3 Navigate nested dictionaries and lists

4 Write Python data as JSON with `json.dump()`

5 Use JSON configuration to control processing behavior

6 Choose between CSV and JSON for a data problem

JSON Is Portable Structured Text

JSON is a data format, not Python source code.

```
{  
  "record_id": "EQ-1001",  
  "status": "active",  
  "error_count": 0  
}
```

JSON file text

```
{  
    "record_id": "EQ-1001",  
    "status": "active",  
    "error_count": 0,  
}
```

Python dictionary literal

Mental model

JSON is a shared language for structured data. Python can translate it into normal dictionaries and lists.

JSON Looks Familiar, But It Has Its Own Rules

A few small differences matter when reading or writing files.

Strings

must use double quotes

`"status"`

Booleans

lowercase

`true / false`

No value

lowercase null

`null`

Comments

not allowed in standard JSON

`// not valid`

Trailing commas not allowed

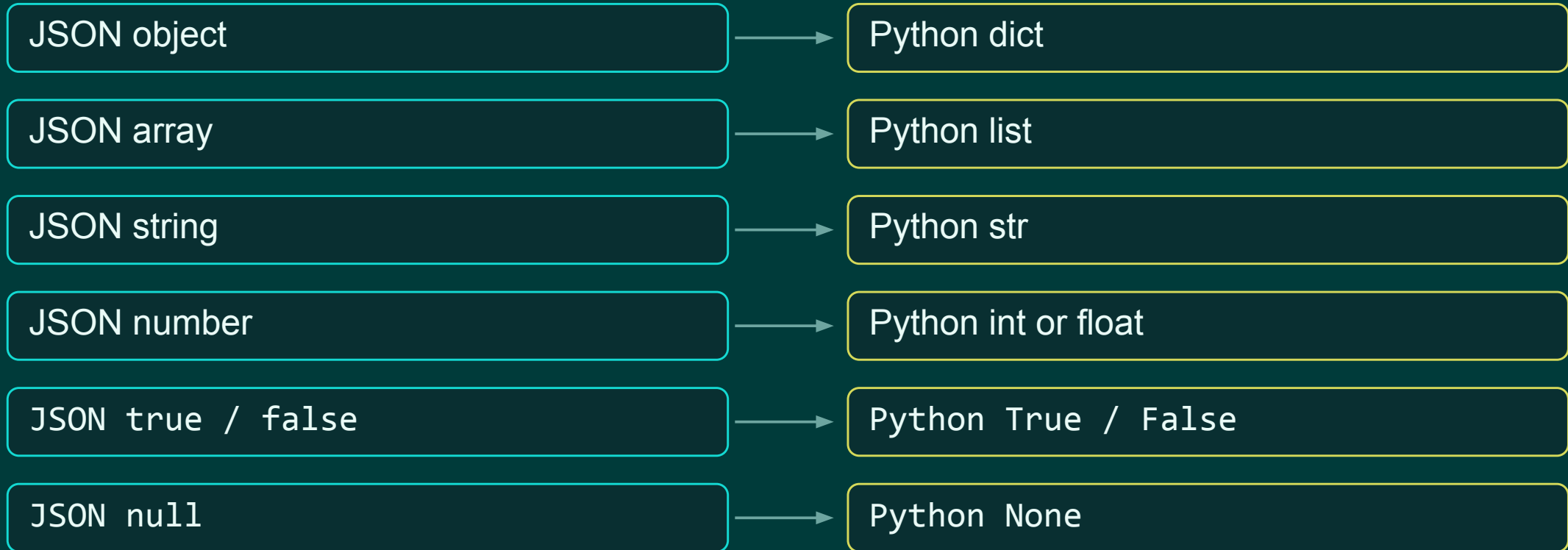
`last item has no comma`

Important

When a JSON file breaks, the problem is often punctuation: quotes, commas, or brackets.

JSON Values Map to Python Values

After loading JSON, you work with normal Python objects.



Translation happens when `json.load()` reads the file.

Use the json Module to Read JSON

Python's standard library includes the translator.

```
import json

with open(
    "data/config.json",
    encoding="utf-8"
) as file:

    config = json.load(file)
```

json.load(file)

Reads JSON text from an open file and converts it into Python data.

Remember

open() gives access to the file.
json.load() interprets the contents.

After Loading, It Is Just Python Data

The file format disappears once the data is in memory.

```
source_system = config["source_system"]  
processing_rate = config["processing_rate"]  
allowed_statuses = config["allowed_statuses"]
```

config is a dictionary

allowed_statuses is a list

normal indexing works

Connection

This is why Day 3 dictionaries and lists matter: JSON commonly becomes those same structures.

A JSON Array Can Become a List of Records

This matches the list-of-dictionaries pattern from Day 3.

```
[
  {
    "record_id": "EQ-1001",
    "status": "active"
  },
  {
    "record_id": "EQ-1002",
    "status": "inactive"
  }
]
```

```
with open(
    "data/records.json",
    encoding="utf-8"
) as file:

    records = json.load(file)

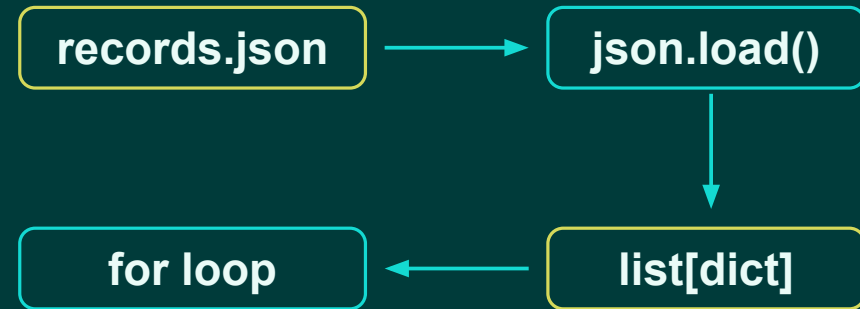
print(type(records))
print(type(records[0]))
```

Conceptual result: records is a list; each item is a dictionary.

Loaded JSON Can Feed Existing Loops

The source changed; the loop pattern did not.

```
for record in records:  
    record_id = record["record_id"]  
    status = record["status"]  
    print(record_id, status)
```



Reuse win

Functions from Day 4 can process JSON-loaded records without knowing the original file format.

Nested JSON Means Nested Python Structures

Objects can contain other objects and arrays.

```
{  
  "record_id": "EQ-1001",  
  "metrics": {  
    "temperature": 72.5,  
    "operating_hours": 850  
  }  
}
```

```
temperature = (  
    data["metrics"]["temperature"]  
)  
  
hours = (  
    data["metrics"]["operating_hours"]  
)
```

Read from the outside inward

First choose the metrics dictionary, then choose the temperature value inside it.

Nested Arrays Are Common in Real Data

JSON can carry collections inside each record.

```
{  
  "record_id": "EQ-1001",  
  "tags": [  
    "urgent",  
    "calibration",  
    "field"  
  ]  
}
```

```
tags = data["tags"]  
  
for tag in tags:  
    print(tag)  
  
first_tag = data["tags"][0]
```

A JSON array becomes a Python list, so indexing and loops work exactly as before.

BLOCK 36

Use `json.dump()` to Write JSON

Create Python data first, then serialize it to a file.

```
import json

summary = {
    "records_processed": 100,
    "records_rejected": 3,
    "success": True,
}
```

```
with open(
    "data/output/summary.json",
    "w",
    encoding="utf-8"
) as file:

    json.dump(summary, file, indent=4)
```

Function pair

`json.load()` reads JSON into Python. `json.dump()` writes Python data out as JSON.

indent=4 Makes Output Human-Readable

The data is the same; the spacing is easier to inspect.

```
json.dump(summary, file)
```

```
{"records_processed":100,"records_rejected":3,"success":true}
```

Harder to read

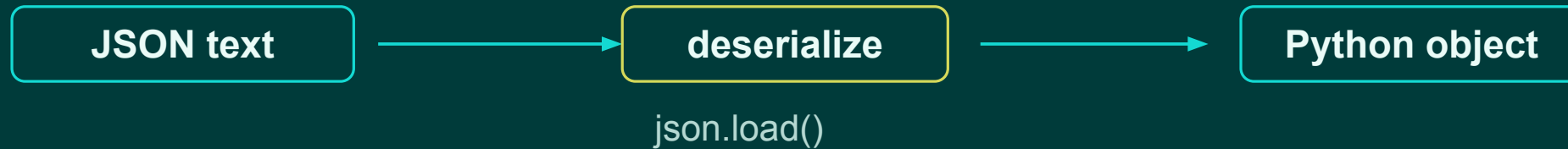
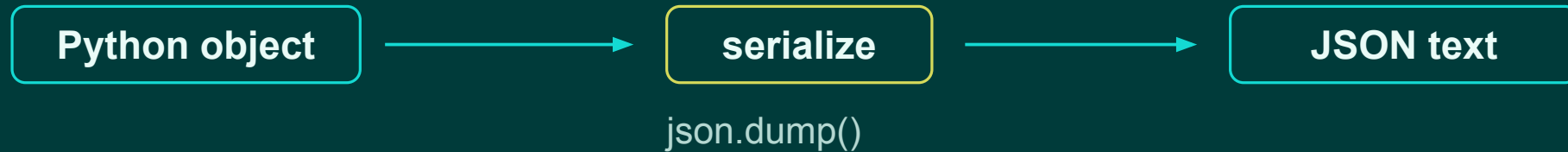
```
json.dump(summary, file, indent=4)
```

```
{  
    "records_processed": 100,  
    "records_rejected": 3,  
    "success": true  
}
```

Easier to inspect

Serialization and Deserialization

These terms simply describe translating between memory and text.



Mental model

JSON is the text representation. Python dictionaries and lists are the in-memory representation.

JSON Is Useful for Configuration

Change settings without editing processing code.

```
{
  "allowed_statuses": [
    "active",
    "inactive",
    "retired"
  ],
  "limits": {
    "maximum_errors": 5
  }
}
```

```
maximum_errors = (
    config["limits"]["maximum_errors"]
)

allowed_statuses = (
    config["allowed_statuses"]
)
```

Why it matters

Rules such as allowed statuses and maximum errors often belong in data, not scattered through code.

CSV Versus JSON

Choose the format that matches the shape of the data.

CSV

Best when data is flat:

- rows and columns
- spreadsheet-like records
- simple tables
- one record per line

JSON

Best when data is structured:

- nested values
- configuration
- API-style messages
- lists inside records

Useful shortcut

CSV $\approx$ table. JSON $\approx$ nested Python collections.

Guided Exercise: 30 Minutes

Students practice loading, reading, using, and writing JSON.

- 1 Load configuration from a JSON file
- 2 Extract values from nested structures
- 3 Use configuration instead of hard-coded constants
- 4 Create a summary dictionary
- 5 Write the summary as `processing_summary.json`

Timeline

Use about 6 minutes per exercise. Instructor circulates during exercises 2 and 3.

Exercise Data: processing_config.json

Save this file before starting the exercises.

```
{
  "source_system": "maintenance",
  "processing_rate": 250,
  "allowed_statuses": [
    "active",
    "inactive",
    "retired"
  ],
  "limits": {
    "maximum_errors": 5,
    "maximum_file_size_mb": 500
  }
}
```

Setup

Place the file in
config/processing_config.j
son.

Reminder

JSON uses true/false/null
when needed, not
True/False/None.

Exercise 1: Load Configuration

Start by opening the file and using `json.load()`.

```
import json
from pathlib import Path

config_path = Path(
    "config/processing_config.json"
)

with open(config_path, encoding="utf-8") as file:
    config = json.load(file)

print(config)
```

Task

Run the file and verify that `config` behaves like a dictionary.

Timer

6 minutes

Exercise 2: Retrieve Values

Practice indexing into dictionaries and nested dictionaries.

```
source_system = config["source_system"]
processing_rate = config["processing_rate"]
allowed_statuses = config["allowed_statuses"]
```

```
maximum_errors = (
    config["limits"]["maximum_errors"]
)
```

```
maximum_file_size_mb = (
    config["limits"]["maximum_file_size_mb"]
)
```

Task

Print each value on its own line.

Timer

6 minutes

Exercise 3: Use Configuration in Logic

Remove a hard-coded rule and read it from JSON instead.

```
# Avoid hard-coding this everywhere
maximum_errors = 5

if error_count > maximum_errors:
    print("Rejected")
```

Before

```
maximum_errors = (
    config["limits"]["maximum_errors"]
)

if error_count > maximum_errors:
    print("Rejected")
```

After

Task

Test with error_count values of 0, 5, and 6.

Exercise 4: Create JSON Output

Build a summary dictionary and write it to disk.

```
summary = {  
    "input_records": 100,  
    "ready_records": 90,  
    "warning_records": 7,  
    "rejected_records": 3,  
}
```

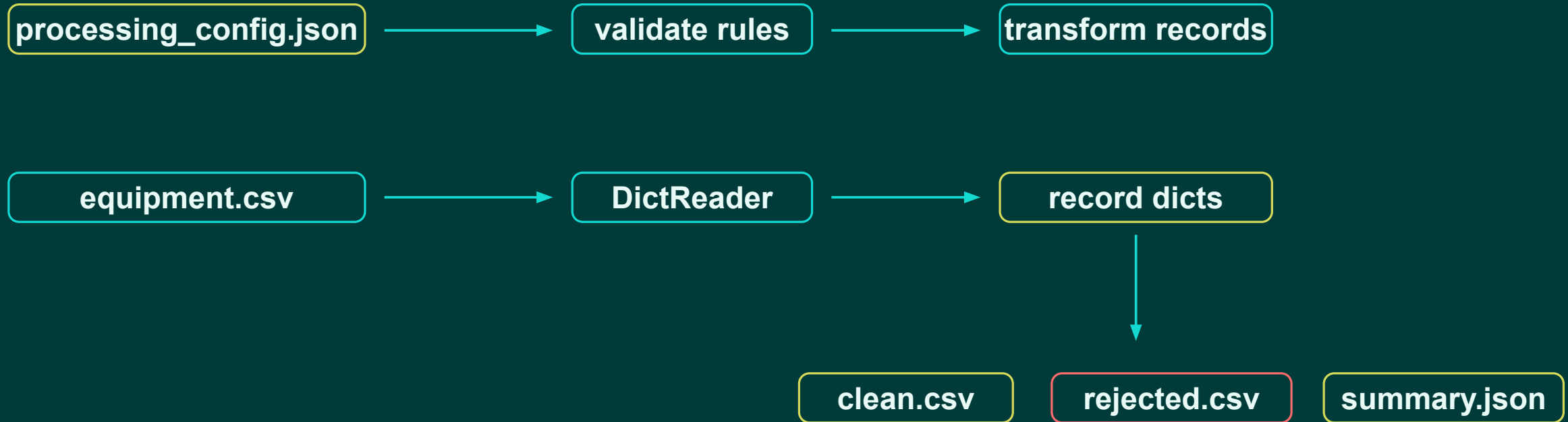
```
with open(  
    "data/output/processing_summary.json",  
    "w",  
    encoding="utf-8"  
) as file:  
  
    json.dump(summary, file, indent=4)
```

Task

Open the output file and inspect the generated JSON.

Applied Challenge: File-Based Data Pipeline

Combine CSV input with JSON configuration and JSON output.



Challenge principle

Configuration controls rules. CSV provides records. Python creates structured outputs.

BLOCK 36

Challenge Inputs

Students receive one CSV file and one JSON configuration file.

data/raw/equipment.csv

```
record_id,status,error_count,completion
EQ-1001,active,0,100
EQ-1002,ACTIVE,2,95
EQ-1003,inactive,0,100
```

config/processing_config.json

```
{
  "allowed_statuses": [
    "active",
    "inactive",
    "retired"
  ],
  "maximum_errors": 5
}
```

Instructor may adapt fields to match the Block 35 CSV processor.

Challenge Requirements

The pipeline should produce clean data, rejected data, and a JSON summary.

1. Load configuration from JSON

2. Read equipment records from CSV

3. Validate required CSV columns

4. Convert numeric values

5. Normalize text values

6. Validate each record with config rules

7. Write clean and rejected CSV files

8. Write summary information to JSON

Expected output

data/output/clean.csv, data/output/rejected.csv, and data/output/summary.json

Block 36 Takeaways

JSON gives Python programs a structured way to exchange data.

Format JSON is structured text, not Python code

Mapping Objects→dicts, arrays→lists, null→None

Reading `json.load()` turns a file into Python data

Writing `json.dump()` serializes Python data to a file

Nested Use repeated indexing to reach nested values

Choice CSV is table-shaped; JSON is structure-shaped

Next Afternoon: imports, modules, and project organization